Cyber Threat Reconstruction Engine: Project Report

What it is

CTRE is an offline tool for incident response. It reads three kinds of evidence that are usually looked at with separate tools, correlates them, and produces one timeline:

- an NTFS `$MFT` (the master file table),
- an NTFS `$UsnJrnl:$J` (the change journal),
- a `.pcap` network capture.

Everything runs on copies of already-collected artifacts. Every input is opened read-only, and the tool needs no third-party packages. You run it as a local web page or from the command line.

Scope

The original brief had a stage that read TLS session keys out of a memory dump and used them to decrypt captured HTTPS traffic. That part is not built. The same code decrypts anyone's traffic given a key store and a capture, so it was left out regardless of how it was packaged. The rest of the workflow does not depend on it.

What the tool does use from the network is metadata that is already visible without any keys: which hosts talked, on which ports, for how long, how many bytes moved and in which direction, DNS query names, and the server name sent in the clear in a TLS ClientHello.

How it works

The pipeline has three parts.

Disk parsing. The `$MFT` parser reads each FILE record, applies the update-sequence fixups, and pulls the timestamps out of the `$STANDARD_INFORMATION` and `$FILE_NAME` attributes. The `$UsnJrnl` parser reads the USN_RECORD_V2 entries, which give an append-only history of what changed and when. The journal file is memory-mapped and the parser skips over the sparse zero regions, so a multi-gigabyte journal does not have to be read into memory.

Timestomping detection. The detector compares the two timestamp sources against each other and against the journal. It flags four things:

- a `$STANDARD_INFORMATION` creation time earlier than the `$FILE_NAME` creation time, which is hard to produce without tampering;
- an SI creation time that predates the file's first journal record, which cannot happen for a file the journal watched being created;
- SI timestamps with their sub-second precision zeroed, which many timestomping tools leave behind;
- an SI modified time newer than any journal entry for the file.

Each result carries a severity and a plain-language reason, so the output is usable in a report instead of being a bare yes or no.

Network metadata and correlation. The pcap parser builds per-connection flow records and reads DNS names and TLS server names. The correlator then looks for a large outbound transfer that lines up in time with a disk change in the journal, which is the shape of a stage-and-exfiltrate step. All of the disk and network events go into one timeline sorted by time.

Output

The tool produces a single HTML report and a JSON export. The report has a findings table, the

correlations, and a timeline you can filter by source. It is a static file with no external assets and everything escaped, so a hostile filename in the evidence cannot inject markup.

Web interface

Running `python -m ctre` with no arguments starts a local server on `127.0.0.1:8765` and opens it in a browser. You drag in the artifacts or load a built-in sample case, set the options, and run the analysis. The results appear in the page, and you can download the report or the JSON.

The server only listens on the loopback address. It checks the `Host` header, requires a custom header on every API call so that other websites open in the browser cannot drive it, sends a strict Content-Security-Policy, and deletes uploaded artifacts when it stops.

Testing

The project ships a generator that writes synthetic artifacts in the real on-disk structures the parsers expect. It plants one timestomped file and one outbound transfer timed to match a disk change, so the detectors and the correlator have something real to find. Because the generator writes the same structures the parsers read, the two check each other.

There are 20 tests. They cover the FILETIME conversion, a round trip through each parser, the detectors, the correlation, report escaping, and a full upload-analyze-download cycle against the running web server over HTTP. Two of the detector tests come from the validation pass: they confirm that a journal that has wrapped, and an MFT record that was reused after a deletion, no longer produce a false timestomping result.

The pcap parser was cross-checked against scapy, a separate library. Packet count, DNS name, TLS server name, and byte totals all matched.

Layout

```
ctre/
  filetime.py     FILETIME and datetime conversion
  mft.py          $MFT record parser
  usnjrnl.py      $UsnJrnl:$J parser
  timestomp.py    timestamp-manipulation detectors
  pcap_meta.py    pcap metadata extractor
  correlate.py    timeline and exfil correlation
  report.py       static HTML report
  service.py      pipeline runner shared by the CLI and web UI
  web.py          local web server
  web/            front-end page (HTML, CSS, JS)
  gen_testdata.py synthetic artifact generator
  cli.py          command-line interface
tests/
  test_pipeline.py  parser, detector, and correlation tests
  test_web.py       web server tests
```

Limits

Everything was tested on generated data, not on a disk image or capture from a real machine. The pcap parser handles the classic libpcap format with Ethernet and IPv4; it does not read pcapng, IPv6, or other link types, and it tells you to convert a pcapng file rather than failing silently. Byte counts in the flow records include frame headers, which is what the scapy check measured against.